

02_bam_comparison_depth

August 31, 2026

```
[1]: import pysam
import pandas as pd
import pyarrow as pa
import argparse
import sys
import re
import matplotlib.pyplot as plt
import seaborn as sns
import gc
import numpy as np
from matplotlib.patches import Patch

from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"

[2]: def parse_args():
    parser = argparse.ArgumentParser(description='Extract specific fields from a
    ↪BAM file into a DataFrame')
    parser.add_argument('-i', '--input', required=True, help='Input BAM file')
    parser.add_argument('-o', '--output', help='Output CSV file (default: print
    ↪to stdout)')
    parser.add_argument('-t', '--tag', default='GX', choices=['GX', 'GN'],
                        help='Tag to extract feature from (GX or GN)')
    parser.add_argument('-s', '--separator', default='\t', help='Field
    ↪separator for output')
    parser.add_argument('-l', '--limit', type=int, help='Limit number of rows
    ↪to process')
    return parser.parse_args()

def extract_feature_from_id(sequence_id):
    """Extract the feature part from the sequence ID."""
    # Expected format: BARCODE:FEATURE:OTHER:PARTS
    parts = sequence_id.split(':')
    if len(parts) >= 2:
        # Extract just the feature name
        feature_with_number = parts[1]
```

```

        # Use regex to extract just the feature name (anything before _dup
        ↪ followed by digits)
        match = re.match(r'([~:]+)(?:_dup\d+)?', feature_with_number)
        if match:
            return match.group(1)
        return "Unknown"

def bam_to_dataframe(input_file, feature_tag='GX', limit=None):
    """Read BAM file and convert specified fields to a DataFrame."""
    data = []
    count = 0

    with pysam.AlignmentFile(input_file, "rb", threads=8) as bam:
        for read in bam:
            if limit is not None and count >= limit:
                break

            sequence_id = read.query_name
            mapping_quality = read.mapping_quality
            chromosome = read.reference_name
            start = read.reference_start # 0-based inclusive
            end = read.reference_end     # 0-based exclusive

            # Extract mismatches from NM tag
            nm = read.get_tag('NM') if read.has_tag('NM') else 0

            # Extract local alignment score
            las = read.get_tag('AS') if read.has_tag('AS') else 0

            # Extract stellarscope posterior probability score
            posterior = read.get_tag('XP') if read.has_tag('XP') else 0

            # Extract feature from GX or GN tag
            feature = read.get_tag(feature_tag) if read.has_tag(feature_tag)
            ↪ else "-"

            # Extract feature from sequence ID
            feature_match = re.search(r'([~:]+):', sequence_id)
            id_feature = feature_match.group(1) if feature_match else "Unknown"

            data.append({
                'sequenceID': sequence_id,
                'chromosome': chromosome,
                'start': start,
                'end': end,
                'feature_in_ID': id_feature,

```

```

        'quality': mapping_quality,
        f'{feature_tag}_tag': feature,
        'mismatches': nm,
        'las': las,
        'posterior': posterior
    })

    count += 1

return pd.DataFrame(data)

def main():
    args = parse_args()

    try:
        df = bam_to_dataframe(args.input, args.tag, args.limit)

        if args.output:
            df.to_csv(args.output, sep=args.separator, index=False)
            print(f"Output written to {args.output}")
        else:
            print(df.to_csv(sep=args.separator, index=False))

    except Exception as e:
        sys.stderr.write(f"Error: {str(e)}\n")
        sys.exit(1)

```

```

[3]: dataset_ids_depth = ["simulated_mm_RA_50kRpC", "simulated_mm_RA_20kRpC",
↪ "simulated_mm_RA_5kRpC"]

```

1 SoloTE upload

```

[ ]: SoloTE_path = "/mnt/transfer/results/SoloTEout_thr0/"

```

```

[ ]: ## Import SoloTE bam with all multi-mappers (OLD simulation)
# sample = "old"
# df_SoloTE_old = bam_to_dataframe(SoloTE_path + dataset_id + "/" + sample + "/"
↪ " + sample + "_SoloTE_temp/" + sample + "_teannotated.bam",
#                               limit=None)
# gc.collect()

```

```

[ ]: ## Import SoloTE bam with all multi-mappers (YOUNG simulation)
# sample = "young"
# df_SoloTE_young = bam_to_dataframe(SoloTE_path + dataset_id + "/" + sample +
↪ "/" + sample + "_SoloTE_temp/" + sample + "_teannotated.bam",
#                               limit=None)

```

```
# gc.collect()
```

```
[ ]: #df_SoloTE_old.to_csv("data/df_SoloTE_old.csv")
df_SoloTE_old = pd.read_csv("data/df_SoloTE_old.csv")
```

```
[ ]: #df_SoloTE_young.to_csv("data/df_SoloTE_young.csv")
df_SoloTE_young = pd.read_csv("data/df_SoloTE_young.csv")
```

2 Stellarscope upload

2.1 Stellarscope Old

2.1.1 Pre EM

Primary alignment obtained with

samtools view -b -F 0x100 old_pseudobulk-tmp_tele.bam > old_pseudobulk_primary_tele.bam

```
[ ]: # df_Stellarscope_old_pre = {}
# sample = "old"
# stellarscope_path = "/mnt/transfer/results/stellarscope_out/"

# for dataset_id in dataset_ids_depth:
#     print(dataset_id)

#     bam_path = f"{stellarscope_path}{dataset_id}_pseudobulk_onestep/{sample}/
#     ↪{sample}_pseudobulk_primary_tele.bam"

#     df_Stellarscope_old_pre[dataset_id] = bam_to_dataframe(bam_path,
#     ↪feature_tag='ZF', limit=None)

#     df_Stellarscope_old_pre[dataset_id].to_csv("data/
#     ↪df_Stellarscope_"+sample+"_"+dataset_id+"_pre_v1.5.csv")

#     gc.collect()
```

2.1.2 Post EM

```
[ ]: # df_Stellarscope_old_bestExclude = {}
# sample = "old"
# stellarscope_path = "/mnt/transfer/results/stellarscope_out/"

# for dataset_id in dataset_ids_depth:
#     print(dataset_id)

#     bam_path = f"{stellarscope_path}{dataset_id}_pseudobulk_onestep/{sample}/
#     ↪{sample}_pseudobulk_bestExclude_vScript.bam"
```

```

#     df_Stellarscope_old_bestExclude[dataset_id] = bam_to_dataframe(bam_path,
↪feature_tag='ZF', limit=None)

#     df_Stellarscope_old_bestExclude[dataset_id]['MAPQ_Pre'] =
↪df_Stellarscope_old_bestExclude[dataset_id]['sequenceID'].map(
#         df_Stellarscope_old_pre[dataset_id].set_index('sequenceID')['quality']
#     )

#     df_Stellarscope_old_bestExclude[dataset_id].to_csv(f"data/
↪df_Stellarscope_{sample}_{dataset_id}_bestExclude_v1.5.csv")

#     print(df_Stellarscope_old_bestExclude[dataset_id].shape)

#     gc.collect()

```

2.1.3 Checkpoint

```
[4]: gc.collect()
```

```
[4]: 0
```

```

[5]: sample = "old"

df_Stellarscope_old_pre = {}
df_Stellarscope_old_bestExclude = {}

for dataset_id in dataset_ids_depth:
    df_Stellarscope_old_pre[dataset_id] = pd.read_csv(
        f"data/df_Stellarscope_{sample}_{dataset_id}_pre_v1.5.csv", index_col=0
    )
    df_Stellarscope_old_bestExclude[dataset_id] = pd.read_csv(
        f"data/df_Stellarscope_{sample}_{dataset_id}_bestExclude_v1.5.csv",
↪index_col=0
    )

print(f"Loaded {len(df_Stellarscope_old_pre)} pre datasets,
↪{len(df_Stellarscope_old_bestExclude)} bestExclude datasets")

```

Loaded 3 pre datasets, 3 bestExclude datasets

```

[ ]: df_Stellarscope_old_pre["simulated_mm_RA_full"] = pd.read_csv("data/
↪df_Stellarscope_old_pre_v1.5.csv")
df_Stellarscope_old_bestExclude["simulated_mm_RA_full"] = pd.read_csv("data/
↪df_Stellarscope_old_bestExclude_v1.5.csv")

```

2.1.4 Filter by posterior

0.5

```
[ ]: df_Stellarscope_old_updated_05 = {}

for dataset_id in df_Stellarscope_old_bestExclude:
    df_Stellarscope_old_updated_05[dataset_id] =
    ↪df_Stellarscope_old_bestExclude[dataset_id].loc[
        df_Stellarscope_old_bestExclude[dataset_id].posterior > 0.5, :
    ].copy()

    print(dataset_id, df_Stellarscope_old_updated_05[dataset_id].shape)
```

```
simulated_mm_RA_50kRpC (19566013, 11)
simulated_mm_RA_20kRpC (8978702, 11)
simulated_mm_RA_5kRpC (2411107, 11)
simulated_mm_RA_full (31701712, 12)
```

0.9

```
[ ]: df_Stellarscope_old_updated_09 = {}

for dataset_id in df_Stellarscope_old_bestExclude:
    df_Stellarscope_old_updated_09[dataset_id] =
    ↪df_Stellarscope_old_bestExclude[dataset_id].loc[
        df_Stellarscope_old_bestExclude[dataset_id].posterior > 0.9, :
    ].copy()

    print(dataset_id, df_Stellarscope_old_updated_09[dataset_id].shape)
```

```
simulated_mm_RA_50kRpC (19542912, 11)
simulated_mm_RA_20kRpC (8968722, 11)
simulated_mm_RA_5kRpC (2409044, 11)
simulated_mm_RA_full (31655261, 12)
```

0.95

```
[ ]: df_Stellarscope_old_updated_095 = {}

for dataset_id in df_Stellarscope_old_bestExclude:
    df_Stellarscope_old_updated_095[dataset_id] =
    ↪df_Stellarscope_old_bestExclude[dataset_id].loc[
        df_Stellarscope_old_bestExclude[dataset_id].posterior > 0.95, :
    ].copy()

    print(dataset_id, df_Stellarscope_old_updated_095[dataset_id].shape)
```

```
simulated_mm_RA_50kRpC (19540276, 11)
simulated_mm_RA_20kRpC (8967490, 11)
simulated_mm_RA_5kRpC (2408636, 11)
simulated_mm_RA_full (31651295, 12)
```

0.99

```
[ ]: df_Stellarscope_old_updated_099 = {}

for dataset_id in df_Stellarscope_old_bestExclude:
    df_Stellarscope_old_updated_099[dataset_id] =
    ↪df_Stellarscope_old_bestExclude[dataset_id].loc[
        df_Stellarscope_old_bestExclude[dataset_id].posterior > 0.99, :
    ].copy()

    print(dataset_id, df_Stellarscope_old_updated_099[dataset_id].shape)
```

```
simulated_mm_RA_50kRpC (19531373, 11)
simulated_mm_RA_20kRpC (8963292, 11)
simulated_mm_RA_5kRpC (2407437, 11)
simulated_mm_RA_full (31638664, 12)
```

2.1.5 Df for plots

```
[ ]: def add_correct_feature(df):
    """Label each read as correctly assigned, incorrectly assigned, or unmapped.
    ↪"""
    df['correct_feature'] = (df.feature_in_ID == df.ZF_tag)
    df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
    return df['correct_feature'].value_counts()

datasets = {
    'pre': df_Stellarscope_old_pre,
    'bestExclude': df_Stellarscope_old_bestExclude,
    'thr_0.5': df_Stellarscope_old_updated_05,
    'thr_0.9': df_Stellarscope_old_updated_09,
    'thr_0.95': df_Stellarscope_old_updated_095,
    'thr_0.99': df_Stellarscope_old_updated_099,
}

accuracy_counts = {
    name: {
        dataset_id: add_correct_feature(df)
        for dataset_id, df in df_dict.items()
    }
    for name, df_dict in datasets.items()
}

sns.set_context("talk")
sns.set_style("white")

def make_crosstab_long(df, quality_col, tool_label, dataset_id):
    """Crosstab correct_feature x quality column, reshape to long format, tag
    ↪with tool name and dataset_id."""
```

```

    data = pd.crosstab(df.correct_feature, df[quality_col])
    data = data.reset_index().melt(id_vars='correct_feature',
    ↪var_name='MappingQuality', value_name='Count')
    data['Tool'] = tool_label
    data['dataset_id'] = dataset_id
    data = data.rename(columns={'correct_feature': 'MappingResult'})
    return data

# name -> (dict of dataframes keyed by dataset_id, quality column name, tool_
    ↪label)
datasets = {
    'pre': (df_Stellarscope_old_pre, 'quality',
    ↪'oldTEs_StellarscopePreEM'),
    'bestExclude': (df_Stellarscope_old_bestExclude, 'MAPQ_Pre',
    ↪'oldTEs_Stellarscope_postEM_bestExclude'),
    'thr_0.5': (df_Stellarscope_old_updated_05, 'MAPQ_Pre',
    ↪'oldTEs_Stellarscope_postEM_thr05'),
    'thr_0.9': (df_Stellarscope_old_updated_09, 'MAPQ_Pre',
    ↪'oldTEs_Stellarscope_postEM_thr09'),
    'thr_0.95': (df_Stellarscope_old_updated_095, 'MAPQ_Pre',
    ↪'oldTEs_Stellarscope_postEM_thr095'),
    'thr_0.99': (df_Stellarscope_old_updated_099, 'MAPQ_Pre',
    ↪'oldTEs_Stellarscope_postEM_thr099'),
}

data_all_Stellarscope_old = pd.concat(
    [
        make_crosstab_long(df, qcol, label, dataset_id)
        for df_dict, qcol, label in datasets.values()
        for dataset_id, df in df_dict.items()
    ],
    ignore_index=True
)

data_all_Stellarscope_old.head()

```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an

item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```

df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future

```

version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
```

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1630477850.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
```

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

```
[ ]: MappingResult MappingQuality    Count      Tool \
0      False              0  176000  oldTEs_StellarscopePreEM
1       True              0   96073  oldTEs_StellarscopePreEM
2      False              1   19506  oldTEs_StellarscopePreEM
3       True              1   49984  oldTEs_StellarscopePreEM
4      False              3   42298  oldTEs_StellarscopePreEM
```

```
dataset_id
0  simulated_mm_RA_50kRpC
1  simulated_mm_RA_50kRpC
2  simulated_mm_RA_50kRpC
3  simulated_mm_RA_50kRpC
4  simulated_mm_RA_50kRpC
```

```
[ ]: data_all_Stellarscope_old.dataset_id.unique()
```

```
data_all_Stellarscope_old
```

```
data_all_Stellarscope_old.to_pickle(
    "data/data_all_Stellarscope_old.pkl"
)
```

```
[ ]: array(['simulated_mm_RA_50kRpC', 'simulated_mm_RA_20kRpC',
          'simulated_mm_RA_5kRpC', 'simulated_mm_RA_full'], dtype=object)
```

```
[ ]: MappingResult MappingQuality    Count \
0      False              0  176000
1       True              0   96073
2      False              1   19506
3       True              1   49984
4      False              3   42298
..      ...              ...      ...
187      True              1   50253
188      False              3  12359
```

189	True	3	166766
190	False	255	76267
191	True	255	31222951

	Tool	dataset_id
0	oldTEs_StellarscopePreEM	simulated_mm_RA_50kRpC
1	oldTEs_StellarscopePreEM	simulated_mm_RA_50kRpC
2	oldTEs_StellarscopePreEM	simulated_mm_RA_50kRpC
3	oldTEs_StellarscopePreEM	simulated_mm_RA_50kRpC
4	oldTEs_StellarscopePreEM	simulated_mm_RA_50kRpC
..	...	...
187	oldTEs_Stellarscope_postEM_thr099	simulated_mm_RA_full
188	oldTEs_Stellarscope_postEM_thr099	simulated_mm_RA_full
189	oldTEs_Stellarscope_postEM_thr099	simulated_mm_RA_full
190	oldTEs_Stellarscope_postEM_thr099	simulated_mm_RA_full
191	oldTEs_Stellarscope_postEM_thr099	simulated_mm_RA_full

[192 rows x 5 columns]

```
[ ]: # --- Map dataset_id -> depth ---
depth_df = pd.DataFrame({
    'dataset_id': ['simulated_mm_RA_full', 'simulated_mm_RA_50kRpC',
    ↪ 'simulated_mm_RA_20kRpC', 'simulated_mm_RA_5kRpC'],          # your existing
    ↪ list, in the order you want
    'depth':      [100, 50, 20, 5]      # replace with your real depth values,
    ↪ same order
})

# --- Compute percentage of correctly assigned reads per dataset_id x Tool ---
pct_correct = (
    data_all_Stellarscope_old
    .groupby(['Tool', 'dataset_id', 'MappingResult'])['Count']
    .sum()
    .reset_index()
)

totals = pct_correct.groupby(['Tool', 'dataset_id'])['Count'].transform('sum')
pct_correct['Percentage'] = pct_correct['Count'] / totals * 100

pct_correct_only = pct_correct[pct_correct['MappingResult'] == True].copy()

# --- Merge in depth ---
pct_correct_only = pct_correct_only.merge(depth_df, on='dataset_id', how='left')

# sort by depth so the line plot draws left-to-right correctly
pct_correct_only = pct_correct_only.sort_values('depth')
```

```

# --- Plot with depth on x-axis ---
plt.figure(figsize=(12, 6))
ax = sns.lineplot(
    data=pct_correct_only,
    x='depth',
    y='Percentage',
    hue='Tool',
    marker='o'
)
plt.ylabel('% Correctly Assigned Reads')
plt.xlabel('Sequencing Depth')
plt.title('Correctly Assigned Reads by Depth and Filtering Strategy')

# force every depth value to appear as a tick
ax.set_xticks(sorted(depth_df['depth'].unique()))
ax.set_xticklabels(sorted(depth_df['depth'].unique()), rotation=45, ha='right')

plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.savefig("figures/correct_assigned_reads_by_depth_old_ratio.pdf",
    ↪bbox_inches='tight')
plt.show()

```

```
[ ]: <Figure size 1200x600 with 0 Axes>
```

```
[ ]: Text(0, 0.5, '% Correctly Assigned Reads')
```

```
[ ]: Text(0.5, 0, 'Sequencing Depth')
```

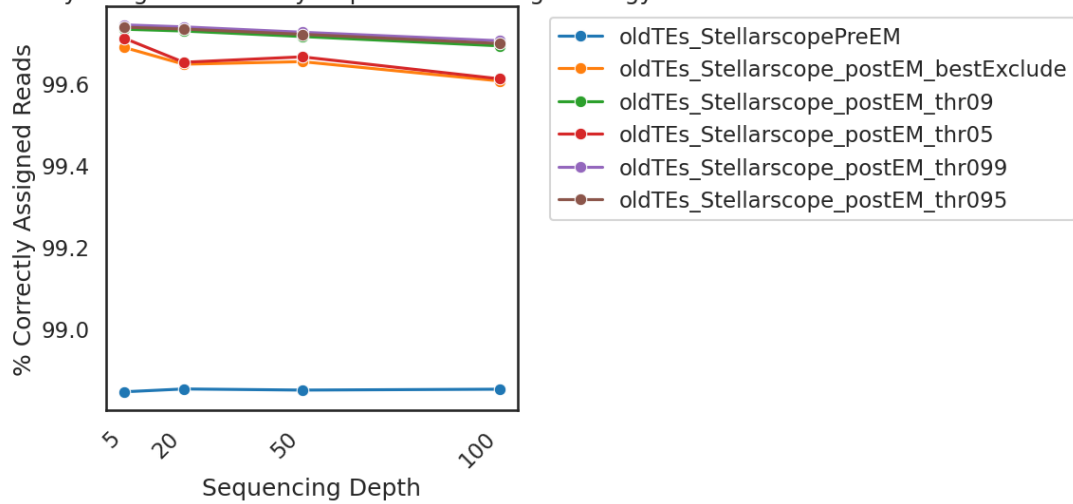
```
[ ]: Text(0.5, 1.0, 'Correctly Assigned Reads by Depth and Filtering Strategy')
```

```
[ ]: [<matplotlib.axis.XTick at 0x748b76017890>,
      <matplotlib.axis.XTick at 0x748b75e4ac10>,
      <matplotlib.axis.XTick at 0x748b75e4b390>,
      <matplotlib.axis.XTick at 0x748b75e4bb10>]
```

```
[ ]: [Text(5, 0, '5'), Text(20, 0, '20'), Text(50, 0, '50'), Text(100, 0, '100')]
```

```
[ ]: <matplotlib.legend.Legend at 0x748b76017250>
```

Correctly Assigned Reads by Depth and Filtering Strategy



```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.patches import Patch

sns.set_context("paper")

# --- Map dataset_id to sequencing depth ---
depth_df = pd.DataFrame({
    'dataset_id': [
        'simulated_mm_RA_full',
        'simulated_mm_RA_50kRpC',
        'simulated_mm_RA_20kRpC',
        'simulated_mm_RA_5kRpC'
    ],
    'depth': [100, 50, 20, 5]
})

# --- Tools to include ---
selected_tools = [
    'oldTES_StellarscopePreEM',
    'oldTES_Stellarscope_postEM_bestExclude',
    'oldTES_Stellarscope_postEM_thr09'
]

# --- Filter tools and add sequencing depth ---
plot_data = (
    data_all_Stellarscope_old
```

```

        .loc[data_all_Stellarscope_old['Tool'].isin(selected_tools)]
        .merge(
            depth_df,
            on='dataset_id',
            how='left',
            validate='many_to_one'
        )
    )

# --- Validate depth mapping ---
if plot_data['depth'].isna().any():
    missing = plot_data.loc[
        plot_data['depth'].isna(),
        'dataset_id'
    ].unique()

    raise ValueError(
        f"Missing depth values for dataset(s): {missing.tolist()}"
    )

# --- Validate requested tools ---
missing_tools = set(selected_tools) - set(plot_data['Tool'].unique())

if missing_tools:
    raise ValueError(
        f"These tools were not found: {sorted(missing_tools)}"
    )

# --- Aggregate counts ---
plot_summary = (
    plot_data
    .groupby(
        ['Tool', 'depth', 'MappingResult'],
        as_index=False,
        observed=True
    )['Count']
    .sum()
)

# --- Convert correct/wrong values to columns ---
pivot = (
    plot_summary
    .pivot_table(
        index=['Tool', 'depth'],
        columns='MappingResult',
        values='Count',
        aggfunc='sum',
    )

```

```

        fill_value=0
    )
    .reindex(columns=[True, False], fill_value=0)
    .rename(columns={
        True: 'Correct',
        False: 'Wrong'
    })
    .reset_index()
)

# --- Ensure all tool-depth combinations are represented ---
depth_order = [100, 50, 20, 5]

full_index = pd.MultiIndex.from_product(
    [selected_tools, depth_order],
    names=['Tool', 'depth']
)

pivot = (
    pivot
    .set_index(['Tool', 'depth'])
    .reindex(full_index, fill_value=0)
    .reset_index()
)

# --- Plot appearance ---
correct_color = '#6dbc90'
wrong_color = '#565656'

# Alpha decreases only for the correct portion
depth_alpha = {
    100: 1.00,
    50: 0.80,
    20: 0.60,
    5: 0.40
}

x = np.arange(len(selected_tools))

total_group_width = 0.80
bar_width = total_group_width / len(depth_order)

fig, ax = plt.subplots(figsize=(12, 7))

# --- Draw grouped, stacked bars ---
for i, depth in enumerate(depth_order):

```



```

depth_values = (
    pivot.loc[pivot['depth'].eq(depth)]
        .set_index('Tool')
        .reindex(selected_tools)
)

correct = depth_values['Correct'].to_numpy()
wrong = depth_values['Wrong'].to_numpy()

positions = (
    x
    - total_group_width / 2
    + bar_width / 2
    + i * bar_width
)

# Correct portion: transparency decreases with depth
ax.bar(
    positions,
    correct,
    width=bar_width,
    color=correct_color,
    alpha=depth_alpha[depth],
    edgecolor='none',
    linewidth=0.5
)

# Wrong portion: always full opacity
ax.bar(
    positions,
    wrong,
    width=bar_width,
    bottom=correct,
    color=wrong_color,
    alpha=1.0,
    edgecolor='none',
    linewidth=0.5
)

# --- Axis formatting ---
ax.set_xticks(x)

ax.set_xticklabels(
    selected_tools,
    rotation=30,
    ha='right'
)

```

```

ax.set_ylabel('Read count', fontsize=14)
ax.set_xlabel('Tool', fontsize=14)

ax.set_title(
    'Mapping Results by Tool and Sequencing Depth',
    fontsize=16,
    pad=20
)

ax.grid(
    axis='y',
    linestyle='--',
    alpha=0.25
)

ax.set_axisbelow(True)

# --- Mapping-result legend ---
mapping_legend = [
    Patch(
        facecolor=correct_color,
        edgecolor='none',
        alpha=1.0,
        label='Correct'
    ),
    Patch(
        facecolor=wrong_color,
        edgecolor='none',
        alpha=1.0,
        label='Wrong'
    )
]

first_legend = ax.legend(
    handles=mapping_legend,
    title='Mapping Result',
    loc='upper left',
    bbox_to_anchor=(1.02, 1.0),
    frameon=True
)

ax.add_artist(first_legend)

# --- Depth legend represented through green opacity ---
depth_legend = [
    Patch(

```

```

        facecolor=correct_color,
        edgecolor='none',
        alpha=depth_alpha[depth],
        label=str(depth)
    )
    for depth in depth_order
]

ax.legend(
    handles=depth_legend,
    title='Sequencing Depth',
    loc='upper left',
    bbox_to_anchor=(1.02, 0.65),
    frameon=True
)

plt.tight_layout()

plt.savefig(
    'figures/mapping_results_selected_tools_depth_comparison.pdf',
    dpi=600,
    bbox_inches='tight'
)

plt.show()

```

```
[ ]: <BarContainer object of 3 artists>
```

```
[ ]: <BarContainer object of 3 artists>
```

```
[ ]: <BarContainer object of 3 artists>
```

```
[ ]: <BarContainer object of 3 artists>
```

```
[ ]: <BarContainer object of 3 artists>
```

```
[ ]: <BarContainer object of 3 artists>
```

```
[ ]: <BarContainer object of 3 artists>
```

```
[ ]: <BarContainer object of 3 artists>
```

```
[ ]: [<matplotlib.axis.XTick at 0x748b45869d10>,
      <matplotlib.axis.XTick at 0x748b4618b390>,
      <matplotlib.axis.XTick at 0x748b4618bb10>]
```

```
[ ]: [Text(0, 0, 'oldTEs_StellarscopePreEM'),
      Text(1, 0, 'oldTEs_Stellarscope_postEM_bestExclude'),
```

```

Text(2, 0, 'oldTEs_Stellarscope_postEM_thr09')]

[ ]: Text(0, 0.5, 'Read count')

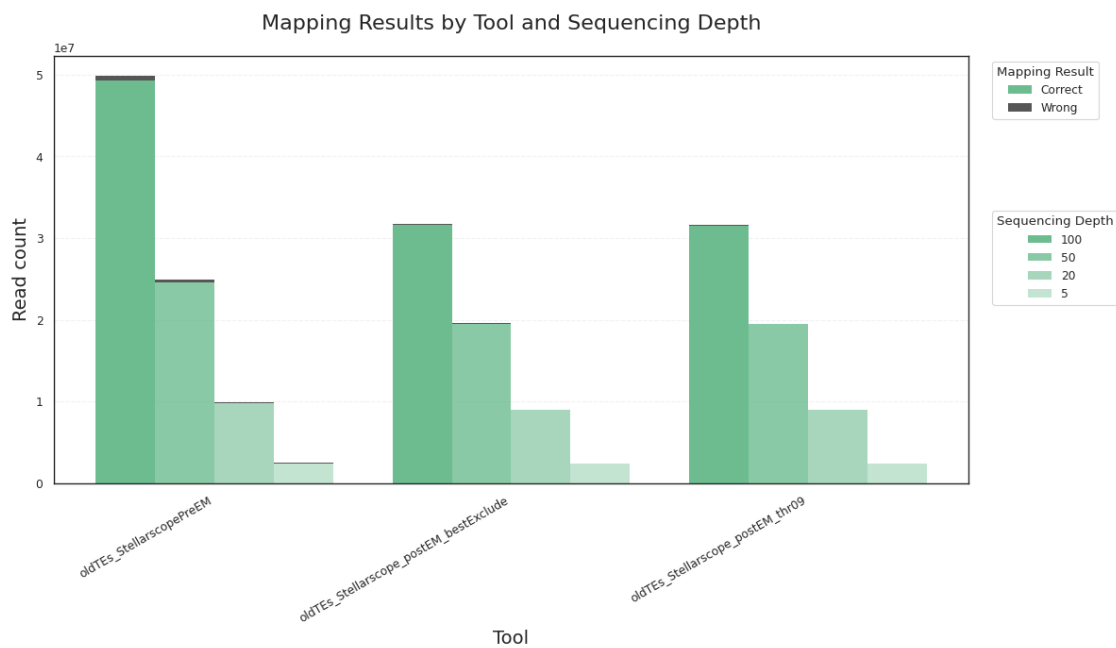
[ ]: Text(0.5, 0, 'Tool')

[ ]: Text(0.5, 1.0, 'Mapping Results by Tool and Sequencing Depth')

[ ]: <matplotlib.legend.Legend at 0x748b45872850>

[ ]: <matplotlib.legend.Legend at 0x748b45873d90>

```



2.2 Stellarscope Young

Get primary mapping

```

samtools view -b -F 0x100 young_pseudobulk-tmp_tele.bam >
young_pseudobulk_primary_tele.bam

```

Best exclude filtering

```

./filter_bam_bestExclude.sh -p young -d /mnt/transfer/results/stellarscope_out/simulated_mm_RA_pseudobulk
2>&1 | tee simulated_mm_RA_young_logfile.txt

```

2.2.1 Pre EM

```
[ ]: # df_Stellarscope_young_pre = {}  
# sample = "young"  
# stellarscope_path = "/mnt/transfer/results/stellarscope_out/"  
  
# for dataset_id in dataset_ids_depth:  
#     print(dataset_id)  
  
#     bam_path = f"{stellarscope_path}{dataset_id}_pseudobulk_onestep/{sample}/  
#     ↪{sample}_pseudobulk_primary_tele.bam"  
  
#     df_Stellarscope_young_pre[dataset_id] = bam_to_dataframe(bam_path, ↪  
#     ↪feature_tag='ZF', limit=None)  
  
#     df_Stellarscope_young_pre[dataset_id].to_csv("data/  
#     ↪df_Stellarscope_\"+sample+\"_\"+dataset_id+\"_pre_v1.5.csv")  
  
#     gc.collect()
```

2.2.2 Post EM

```
[ ]: # df_Stellarscope_young_bestExclude = {}  
# sample = "young"  
# stellarscope_path = "/mnt/transfer/results/stellarscope_out/"  
  
# for dataset_id in dataset_ids_depth:  
#     print(dataset_id)  
  
#     bam_path = f"{stellarscope_path}{dataset_id}_pseudobulk_onestep/{sample}/  
#     ↪{sample}_pseudobulk_bestExclude_vScript.bam"  
  
#     df_Stellarscope_young_bestExclude[dataset_id] = ↪  
#     ↪bam_to_dataframe(bam_path, feature_tag='ZF', limit=None)  
  
#     df_Stellarscope_young_bestExclude[dataset_id]['MAPQ_Pre'] = ↪  
#     ↪df_Stellarscope_young_bestExclude[dataset_id]['sequenceID'].map(  
#         df_Stellarscope_young_bestExclude[dataset_id].  
#         ↪set_index('sequenceID')['quality']  
#     )  
#     df_Stellarscope_young_bestExclude[dataset_id]['MAPQ_Pre'] = ↪  
#     ↪df_Stellarscope_young_bestExclude[dataset_id]['sequenceID'].  
#     ↪map(df_Stellarscope_young_pre[dataset_id].set_index('sequenceID')['quality'])  
  
#     df_Stellarscope_young_bestExclude[dataset_id].to_csv(f"data/  
#     ↪df_Stellarscope_{sample}_{dataset_id}_bestExclude_v1.5.csv")
```

```
#     print(df_Stellarscope_young_bestExclude[dataset_id].shape)

#     gc.collect()
```

2.2.3 Checkpoint

```
[ ]: gc.collect()
```

```
[ ]: 866
```

```
[ ]: sample = "young"

df_Stellarscope_young_pre = {}
df_Stellarscope_young_bestExclude = {}

for dataset_id in dataset_ids_depth:
    df_Stellarscope_young_pre[dataset_id] = pd.read_csv(
        f"data/df_Stellarscope_{sample}_{dataset_id}_pre_v1.5.csv", index_col=0
    )
    df_Stellarscope_young_bestExclude[dataset_id] = pd.read_csv(
        f"data/df_Stellarscope_{sample}_{dataset_id}_bestExclude_v1.5.csv",
        ↪index_col=0
    )

print(f"Loaded {len(df_Stellarscope_young_pre)} pre datasets,
      ↪{len(df_Stellarscope_young_bestExclude)} bestExclude datasets")
```

Loaded 3 pre datasets, 3 bestExclude datasets

```
[ ]: df_Stellarscope_young_pre["simulated_mm_RA_full"] = pd.read_csv("data/
      ↪df_Stellarscope_young_pre_v1.5.csv")
df_Stellarscope_young_bestExclude["simulated_mm_RA_full"] = pd.read_csv("data/
      ↪df_Stellarscope_young_bestExclude_v1.5.csv")
```

2.2.4 Filter by posterior

0.5

```
[ ]: df_Stellarscope_young_updated_05 = {}

for dataset_id in df_Stellarscope_young_bestExclude:
    df_Stellarscope_young_updated_05[dataset_id] =
    ↪df_Stellarscope_young_bestExclude[dataset_id].loc[
        df_Stellarscope_young_bestExclude[dataset_id].posterior > 0.5, :
    ].copy()

    print(dataset_id, df_Stellarscope_young_updated_05[dataset_id].shape)
```

```
simulated_mm_RA_50kRpC (12246109, 11)
simulated_mm_RA_20kRpC (5197017, 11)
simulated_mm_RA_5kRpC (1300293, 11)
simulated_mm_RA_full (21394107, 12)
```

0.9

```
[ ]: df_Stellarscope_young_updated_09 = {}

for dataset_id in df_Stellarscope_young_bestExclude:
    df_Stellarscope_young_updated_09[dataset_id] = _
    ↪df_Stellarscope_young_bestExclude[dataset_id].loc[
        df_Stellarscope_young_bestExclude[dataset_id].posterior > 0.9, :
    ].copy()

    print(dataset_id, df_Stellarscope_young_updated_09[dataset_id].shape)
```

```
simulated_mm_RA_50kRpC (9503201, 11)
simulated_mm_RA_20kRpC (3904970, 11)
simulated_mm_RA_5kRpC (921362, 11)
simulated_mm_RA_full (16377089, 12)
```

0.95

```
[ ]: df_Stellarscope_young_updated_095 = {}

for dataset_id in df_Stellarscope_young_bestExclude:
    df_Stellarscope_young_updated_095[dataset_id] = _
    ↪df_Stellarscope_young_bestExclude[dataset_id].loc[
        df_Stellarscope_young_bestExclude[dataset_id].posterior > 0.95, :
    ].copy()

    print(dataset_id, df_Stellarscope_young_updated_095[dataset_id].shape)
```

```
simulated_mm_RA_50kRpC (8488411, 11)
simulated_mm_RA_20kRpC (3560363, 11)
simulated_mm_RA_5kRpC (828645, 11)
simulated_mm_RA_full (14964159, 12)
```

0.99

```
[ ]: df_Stellarscope_young_updated_099 = {}

for dataset_id in df_Stellarscope_young_bestExclude:
    df_Stellarscope_young_updated_099[dataset_id] = _
    ↪df_Stellarscope_young_bestExclude[dataset_id].loc[
        df_Stellarscope_young_bestExclude[dataset_id].posterior > 0.99, :
    ].copy()

    print(dataset_id, df_Stellarscope_young_updated_099[dataset_id].shape)
```

```

simulated_mm_RA_50kRpC (7092698, 11)
simulated_mm_RA_20kRpC (2841109, 11)
simulated_mm_RA_5kRpC (672194, 11)
simulated_mm_RA_full (12547924, 12)

```

2.2.5 Plot % of correctly assigned reads by depth

```

[ ]: def add_correct_feature(df):
    """Label each read as correctly assigned, incorrectly assigned, or unmapped.
    ↪ """
    df['correct_feature'] = (df.feature_in_ID == df.ZF_tag)
    df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
    return df['correct_feature'].value_counts()

datasets = {
    'pre': df_Stellarscope_young_pre,
    'bestExclude': df_Stellarscope_young_bestExclude,
    'thr_0.5': df_Stellarscope_young_updated_05,
    'thr_0.9': df_Stellarscope_young_updated_09,
    'thr_0.95': df_Stellarscope_young_updated_095,
    'thr_0.99': df_Stellarscope_young_updated_099,
}

accuracy_counts = {
    name: {
        dataset_id: add_correct_feature(df)
        for dataset_id, df in df_dict.items()
    }
    for name, df_dict in datasets.items()
}

sns.set_context("talk")
sns.set_style("white")

def make_crosstab_long(df, quality_col, tool_label, dataset_id):
    """Crosstab correct_feature x quality column, reshape to long format, tag_
    ↪ with tool name and dataset_id."""
    data = pd.crosstab(df.correct_feature, df[quality_col])
    data = data.reset_index().melt(id_vars='correct_feature',
    ↪ var_name='MappingQuality', value_name='Count')
    data['Tool'] = tool_label
    data['dataset_id'] = dataset_id
    data = data.rename(columns={'correct_feature': 'MappingResult'})
    return data

# name -> (dict of dataframes keyed by dataset_id, quality column name, tool_
    ↪ label)

```



```

datasets = {
    'pre': (df_Stellarscope_young_pre, 'quality', □
    ↪ 'youngTEs_StellarscopePreEM'),
    'bestExclude': (df_Stellarscope_young_bestExclude, 'MAPQ_Pre', □
    ↪ 'youngTEs_Stellarscope_postEM_bestExclude'),
    'thr_0.5': (df_Stellarscope_young_updated_05, 'MAPQ_Pre', □
    ↪ 'youngTEs_Stellarscope_postEM_thr05'),
    'thr_0.9': (df_Stellarscope_young_updated_09, 'MAPQ_Pre', □
    ↪ 'youngTEs_Stellarscope_postEM_thr09'),
    'thr_0.95': (df_Stellarscope_young_updated_095, 'MAPQ_Pre', □
    ↪ 'youngTEs_Stellarscope_postEM_thr095'),
    'thr_0.99': (df_Stellarscope_young_updated_099, 'MAPQ_Pre', □
    ↪ 'youngTEs_Stellarscope_postEM_thr099'),
}

data_all_Stellarscope_young = pd.concat(
    [
        make_crosstab_long(df, qcol, label, dataset_id)
        for df_dict, qcol, label in datasets.values()
        for dataset_id, df in df_dict.items()
    ],
    ignore_index=True
)

data_all_Stellarscope_young.head()

```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an

item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```

df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future
version of pandas. Value 'Unmapped' has dtype incompatible with bool, please
explicitly cast to a compatible dtype first.
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
/mnt/transfer/tmp/ipykernel_3478992/1001192103.py:4: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a future

```

version of pandas. Value 'Unmapped' has dtype incompatible with bool, please explicitly cast to a compatible dtype first.

```
df.loc[df.ZF_tag == "-", 'correct_feature'] = "Unmapped"
```

```
[ ]: MappingResult MappingQuality      Count      Tool \
0      False              0  7541259  youngTEs_StellarscopePreEM
1       True              0  3138346  youngTEs_StellarscopePreEM
2      False              1   541548  youngTEs_StellarscopePreEM
3       True              1   689794  youngTEs_StellarscopePreEM
4      False              3   689027  youngTEs_StellarscopePreEM

      dataset_id
0  simulated_mm_RA_50kRpC
1  simulated_mm_RA_50kRpC
2  simulated_mm_RA_50kRpC
3  simulated_mm_RA_50kRpC
4  simulated_mm_RA_50kRpC
```

2.2.6 Checkpoint

```
[ ]: gc.collect()
```

```
[ ]: 0
```

```
[ ]: # Save
# data_all_Stellarscope_young.to_parquet(
#     "data/data_all_Stellarscope_young.parquet",
#     index=False
# )
# Reimport
data_all_Stellarscope_young = pd.read_parquet(
    "data/data_all_Stellarscope_young.parquet"
)
```

```
[ ]: sns.set_context("talk")

# --- Map dataset_id -> depth ---
depth_df = pd.DataFrame({
    'dataset_id': ['simulated_mm_RA_5kRpC',
                  'simulated_mm_RA_20kRpC',
                  'simulated_mm_RA_50kRpC',
                  'simulated_mm_RA_full'],
    'depth':      [5, 20, 50, 100]
})

# --- Compute percentage of correctly assigned reads per dataset_id x Tool ---
pct_correct = (
    data_all_Stellarscope_young
```

```

        .groupby(['Tool', 'dataset_id', 'MappingResult'])['Count']
        .sum()
        .reset_index()
    )

    totals = pct_correct.groupby(['Tool', 'dataset_id'])['Count'].transform('sum')
    pct_correct['Percentage'] = pct_correct['Count'] / totals * 100

    pct_correct_only = pct_correct[pct_correct['MappingResult'] == True].copy()

    # --- Merge in depth ---
    pct_correct_only = pct_correct_only.merge(depth_df, on='dataset_id', how='left')

    # sort by depth so the line plot draws left-to-right correctly
    pct_correct_only = pct_correct_only.sort_values('depth')

    # --- Plot with depth on x-axis ---
    plt.figure(figsize=(12, 6))
    ax = sns.lineplot(
        data=pct_correct_only,
        x='depth',
        y='Percentage',
        hue='Tool',
        marker='o'
    )
    plt.ylabel('% Correctly Assigned Reads')
    plt.xlabel('Sequencing Depth')
    plt.title('Correctly Assigned Reads by Depth and Filtering Strategy')

    # force every depth value to appear as a tick
    ax.set_xticks(sorted(depth_df['depth'].unique()))
    ax.set_xticklabels(sorted(depth_df['depth'].unique()), rotation=45, ha='right')

    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.tight_layout()
    plt.savefig("figures/correct_assigned_reads_by_depth_young.pdf",
        ↪bbox_inches='tight')
    plt.show()

```

```
[ ]: <Figure size 1200x600 with 0 Axes>
```

```
[ ]: Text(0, 0.5, '% Correctly Assigned Reads')
```

```
[ ]: Text(0.5, 0, 'Sequencing Depth')
```

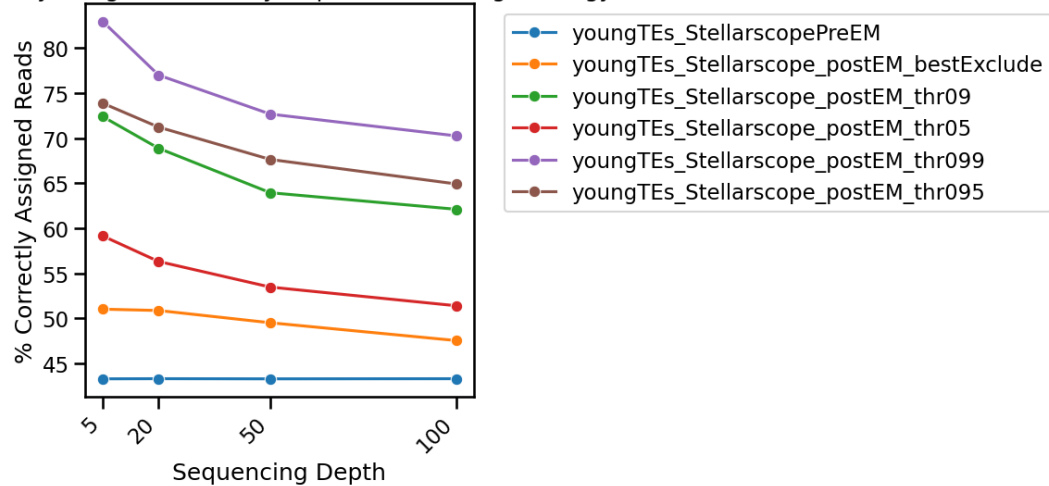
```
[ ]: Text(0.5, 1.0, 'Correctly Assigned Reads by Depth and Filtering Strategy')
```

```
[ ]: [<matplotlib.axis.XTick at 0x705e4bd2cad0>,
      <matplotlib.axis.XTick at 0x705e4bd09590>,
      <matplotlib.axis.XTick at 0x705e4bd3afd0>,
      <matplotlib.axis.XTick at 0x705e4bd3b750>]

[ ]: [Text(5, 0, '5'), Text(20, 0, '20'), Text(50, 0, '50'), Text(100, 0, '100')]

[ ]: <matplotlib.legend.Legend at 0x705e482b4e10>
```

Correctly Assigned Reads by Depth and Filtering Strategy



```
[ ]: sns.set_context("paper")

# --- Map dataset_id to sequencing depth ---
depth_df = pd.DataFrame({
    'dataset_id': [
        'simulated_mm_RA_full',
        'simulated_mm_RA_50kRpC',
        'simulated_mm_RA_20kRpC',
        'simulated_mm_RA_5kRpC'
    ],
    'depth': [100, 50, 20, 5]
})

# --- Tools to include ---
selected_tools = [
    'youngTEs_StellarscopePreEM',
    'youngTEs_Stellarscope_postEM_bestExclude',
    'youngTEs_Stellarscope_postEM_thr05',
    'youngTEs_Stellarscope_postEM_thr09',
    'youngTEs_Stellarscope_postEM_thr095',
    'youngTEs_Stellarscope_postEM_thr099'
```

```

]

# --- Filter tools and add sequencing depth ---
plot_data = (
    data_all_Stellarscope_young
    .loc[data_all_Stellarscope_young['Tool'].isin(selected_tools)]
    .merge(
        depth_df,
        on='dataset_id',
        how='left',
        validate='many_to_one'
    )
)

# --- Validate depth mapping ---
if plot_data['depth'].isna().any():
    missing = plot_data.loc[
        plot_data['depth'].isna(),
        'dataset_id'
    ].unique()

    raise ValueError(
        f"Missing depth values for dataset(s): {missing.tolist()}"
    )

# --- Validate requested tools ---
missing_tools = set(selected_tools) - set(plot_data['Tool'].unique())

if missing_tools:
    raise ValueError(
        f"These tools were not found: {sorted(missing_tools)}"
    )

# --- Aggregate counts ---
plot_summary = (
    plot_data
    .groupby(
        ['Tool', 'depth', 'MappingResult'],
        as_index=False,
        observed=True
    )['Count']
    .sum()
)

# --- Convert correct/wrong values to columns ---
pivot = (
    plot_summary

```

```

        .pivot_table(
            index=['Tool', 'depth'],
            columns='MappingResult',
            values='Count',
            aggfunc='sum',
            fill_value=0
        )
        .reindex(columns=[True, False], fill_value=0)
        .rename(columns={
            True: 'Correct',
            False: 'Wrong'
        })
        .reset_index()
    )

    # --- Ensure all tool-depth combinations are represented ---
    depth_order = [5, 20, 50, 100]

    full_index = pd.MultiIndex.from_product(
        [selected_tools, depth_order],
        names=['Tool', 'depth']
    )

    pivot = (
        pivot
        .set_index(['Tool', 'depth'])
        .reindex(full_index, fill_value=0)
        .reset_index()
    )

    # --- Plot appearance ---
    correct_color = '#6dbc90'
    wrong_color = '#565656'

    # Alpha decreases only for the correct portion
    depth_alpha = {
        100: 1.00,
        50: 0.70,
        20: 0.50,
        5: 0.30
    }

    x = np.arange(len(selected_tools))

    total_group_width = 0.80
    bar_width = total_group_width / len(depth_order)

```



```

fig, ax = plt.subplots(figsize=(12, 7))

# --- Draw grouped, stacked bars ---
for i, depth in enumerate(depth_order):

    depth_values = (
        pivot.loc[pivot['depth'].eq(depth)]
        .set_index('Tool')
        .reindex(selected_tools)
    )

    correct = depth_values['Correct'].to_numpy()
    wrong = depth_values['Wrong'].to_numpy()

    positions = (
        x
        - total_group_width / 2
        + bar_width / 2
        + i * bar_width
    )

    # Correct portion: transparency decreases with depth
    ax.bar(
        positions,
        correct,
        width=bar_width,
        color=correct_color,
        alpha=depth_alpha[depth],
        edgecolor='none',
        linewidth=0.5
    )

    # Wrong portion: always full opacity
    ax.bar(
        positions,
        wrong,
        width=bar_width,
        bottom=correct,
        color=wrong_color,
        alpha=1.0,
        edgecolor='none',
        linewidth=0.5
    )

# --- Axis formatting ---
ax.set_xticks(x)

```

```

ax.set_xticklabels(
    selected_tools,
    rotation=30,
    ha='right'
)

ax.set_ylabel('Read count', fontsize=14)
ax.set_xlabel('Tool', fontsize=14)

ax.set_title(
    'Mapping Results by Tool and Sequencing Depth',
    fontsize=16,
    pad=20
)

ax.grid(
    axis='y',
    linestyle='--',
    alpha=0.25
)

ax.set_axisbelow(True)

# --- Mapping-result legend ---
mapping_legend = [
    Patch(
        facecolor=correct_color,
        edgecolor='none',
        alpha=1.0,
        label='Correct'
    ),
    Patch(
        facecolor=wrong_color,
        edgecolor='none',
        alpha=1.0,
        label='Wrong'
    )
]

first_legend = ax.legend(
    handles=mapping_legend,
    title='Mapping Result',
    loc='upper left',
    bbox_to_anchor=(1.02, 1.0),
    frameon=True
)

```

```

ax.add_artist(first_legend)

# --- Depth legend represented through green opacity ---
depth_legend = [
    Patch(
        facecolor=correct_color,
        edgecolor='none',
        alpha=depth_alpha[depth],
        label=str(depth)
    )
    for depth in depth_order
]

ax.legend(
    handles=depth_legend,
    title='Sequencing Depth',
    loc='upper left',
    bbox_to_anchor=(1.02, 0.65),
    frameon=True
)

plt.tight_layout()

plt.savefig(
    'figures/mapping_results_selected_tools_depth_comparison.pdf',
    dpi=600,
    bbox_inches='tight'
)

plt.show()

```

[]: <BarContainer object of 6 artists>

[]: <BarContainer object of 6 artists>

[]: <BarContainer object of 6 artists>

[]: <BarContainer object of 6 artists>

[]: <BarContainer object of 6 artists>

[]: <BarContainer object of 6 artists>

[]: <BarContainer object of 6 artists>

[]: <BarContainer object of 6 artists>

```
[ ]: [<matplotlib.axis.XTick at 0x705e23576210>,
      <matplotlib.axis.XTick at 0x705e235b5310>,
      <matplotlib.axis.XTick at 0x705e235b5a90>,
      <matplotlib.axis.XTick at 0x705e235b6210>,
      <matplotlib.axis.XTick at 0x705e235b6990>,
      <matplotlib.axis.XTick at 0x705e235b7110>]

[ ]: [Text(0, 0, 'youngTES_StellarscopePreEM'),
      Text(1, 0, 'youngTES_Stellarscope_postEM_bestExclude'),
      Text(2, 0, 'youngTES_Stellarscope_postEM_thr05'),
      Text(3, 0, 'youngTES_Stellarscope_postEM_thr09'),
      Text(4, 0, 'youngTES_Stellarscope_postEM_thr095'),
      Text(5, 0, 'youngTES_Stellarscope_postEM_thr099')]

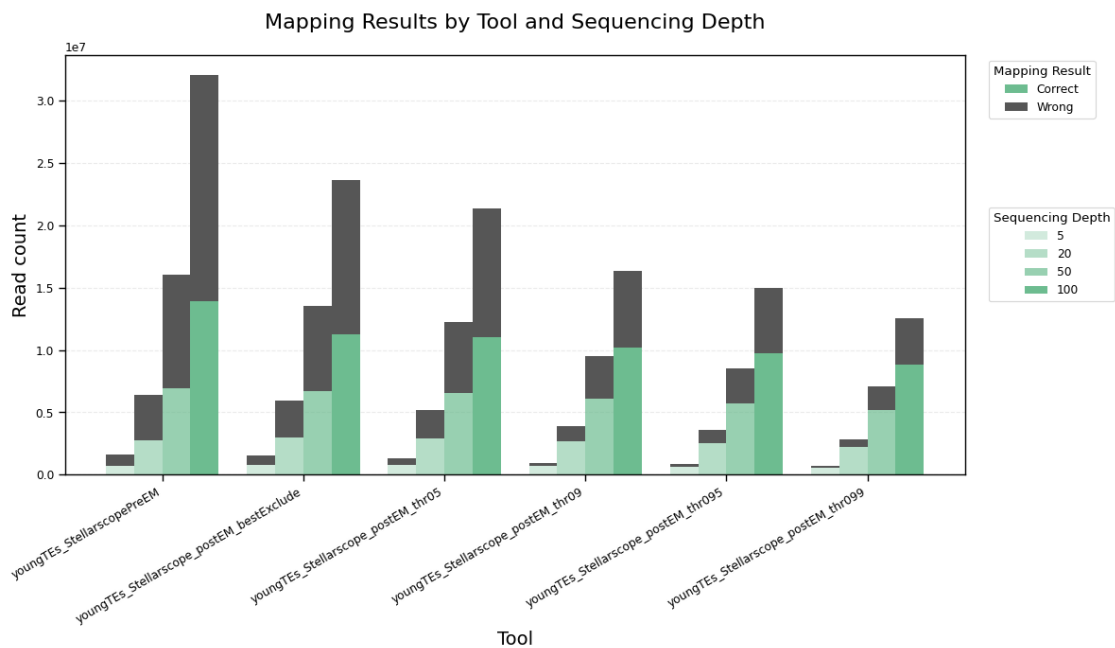
[ ]: Text(0, 0.5, 'Read count')

[ ]: Text(0.5, 0, 'Tool')

[ ]: Text(0.5, 1.0, 'Mapping Results by Tool and Sequencing Depth')

[ ]: <matplotlib.legend.Legend at 0x705e2354df90>

[ ]: <matplotlib.legend.Legend at 0x705e2354f610>
```



```
[ ]:
```